MANCHESTER 1824
The University of Manchester

A Low-Cost Real-Time Haptic Rendering System with Physics-Based Simulation and Force Feedback

A dissertation submitted to The University of Manchester for the degree of

Bachelor of Engineering in Mechatronic Engineering

in the Faculty of Science and Engineering

Year of submission

2026

Student ID

11349510

School of Engineering

Contents

Abstract . . . . . **iv**

Declaration of originality . . . . . **v**

Copyright statement . . . . . **vi**

Acknowledgments . . . . . **vii**

1 Introduction **1**

1.1 Background and Motivation 1

1.2 Problem Definition 1

1.3 Aim and Scope of This Dissertation 2

1.4 Objectives 2

1.5 Contributions of This Work 3

2 Literature review **4**

2.1 Haptic Interfaces for Robotics 4

2.2 Haptic Rendering Techniques 5

2.3 Stability in Haptic Systems 6

2.4 Communication Timing in Real-Time Haptic Loops 6

2.5 Low-Cost Haptic Hardware 7

2.6 Summary of Literature 7

3 System Design and Implementation **8**

3.1 System Overview 8

3.2 Mechanical and Embedded Platform 9

3.3 Communication Protocol and Device Interface 11

3.4 Host Software Architecture 11

3.4.1 World and Rendering Subsystems 12

3.4.2 Haptic Rendering Engine 12

3.4.3 Physics Simulation Integration 13

3.5 Logging and Communication Validation 14

4 Experimental Methodology **14**

4.1 Communication Performance Tests 14

4.1.1 Packet Integrity and Rate Tests 15

4.1.2 Round-Trip Latency Test . . . 15
4.1.3 End-to-End Host-Side Timing Test . . . 15
4.2 Simulation Validation . . . 16
4.2.1 Virtual Wall Test . . . 16
4.2.2 Free-Space and Saturation Checks . . . 16
4.3 Safety Validation . . . 17
4.3.1 Torque Saturation Test . . . 17
4.3.2 Rate Limiting Test . . . 17
4.3.3 Watchdog Timeout Test . . . 17
4.4 Hardware Validation . . . 17
4.4.1 Repeated Contact Test . . . 18
4.4.2 Constrained Motion Hardware Test . . . 18

5 Results . . . 18
5.1 Communication Performance . . . 19
5.1.1 Packet Rate and Integrity Results . . . 19
5.1.2 Round-Trip Latency Results . . . 20
5.1.3 End-to-End Host-Side Timing Results . . . 21
5.2 Simulation Validation Results . . . 23
5.2.1 Virtual Wall Contact Response . . . 23
5.2.2 Force–Penetration Characterisation . . . 24
5.2.3 Free-Space and Saturation Behaviour . . . 25
5.3 Safety Validation Results . . . 25
5.3.1 Torque Saturation . . . 25
5.3.2 Rate Limiting . . . 25
5.3.3 Watchdog Timeout Behaviour . . . 26
5.3.4 Command Consistency . . . 27
5.4 Hardware Validation Results . . . 27
5.4.1 Repeated Contact With Virtual Wall . . . 27
5.4.2 Constrained Motion Along a Surface . . . 28

6 Discussion . . . 29
6.1 Communication and Latency Constraints . . . 29
6.2 Local Contact State Limitation . . . 30
6.3 Observed Contact Behaviour Under Delay . . . 30

6.4 Hardware Limitations . . . 31
6.5 Design Trade-Offs . . . 31
6.6 Project Management and Development Process . . . 32

7 Conclusions and Future Work . . . 33
7.1 Conclusions . . . 33
7.2 Future Work . . . 34

References . . . 36

Appendices . . . 39

A Preliminary Project Proposal . . . 39

B Gantt Chart . . . 40

C Risk Register and Health & Safety Risk Assessment . . . 41

D CPD Log . . . 42

E Mechanical Design Drawing . . . 43

F Source Code and Repository Information . . . 44

Abstract

This dissertation presents the design, implementation, and evaluation of a low-cost haptic rendering system that combines a two-degree-of-freedom physical interface with custom simulation and visualisation software. The work addresses bounded bidirectional haptic interaction on a low-cost platform where actuator capability, communication delay, and transmission behaviour limit achievable fidelity. The system integrates embedded torque-command actuation, host-device communication, physics-based simulation, and proxy-based haptic rendering in a multi-threaded architecture. Although selected subsystems achieved nominal one kilohertz operation with low host-side latency, a round-trip communication delay of approximately 16 ms prevented true end-to-end kilohertz interaction. Simulation tests showed the command-side force model reproduced configured virtual coupling stiffness within five per cent; motor-enabled tests showed bounded contact behaviour in tested interactions. Final physical validation was limited to single-axis force feedback following actuator failure. No calibrated force sensing or current measurement was available. The main finding is that communication architecture and actuator capability, rather than host computation alone, determine achievable stiffness, responsiveness, and force fidelity on this low-cost prototype.

Declaration of originality

I hereby confirm that no portion of the work referred to in the thesis has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright statement

i The author of this thesis (including any appendices and/or schedules to this thesis) owns certain copyright or related rights in it (the "Copyright") and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.

ii Copies of this thesis, either in full or in extracts and whether in hard or electronic copy, may be made *only* in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has from time to time. This page must form part of any such copies made.

iii The ownership of certain Copyright, patents, designs, trademarks and other intellectual property (the "Intellectual Property") and any reproductions of copyright works in the thesis, for example graphs and tables ("Reproductions"), which may be described in this thesis, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.

iv Further information on the conditions under which disclosure, publication and commercialisation of this thesis, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see `http://documents.manchester.ac.uk/DocuInfo.aspx?DocID=24420`), in any relevant Thesis restriction declarations deposited in the University Library, The University Library's regulations (see `http://www.library.manchester.ac.uk/about/regulations/`) and in The University's policy on Presentation of Theses.

Acknowledgments

Use of AI Tools

Large language models (including ChatGPT) were used during this project as a supplementary tool to assist with language refinement, report structuring, and code debugging.

These tools were used to improve clarity, grammar, and organisation of written content, and to support troubleshooting during software development.

All technical content, system design, implementation decisions, analysis, and conclusions are my own work. All material included in this report has been reviewed and verified by me.

1 Introduction

This section introduces the project motivation, problem definition, objectives, contributions, and scope limitations.

1.1 Background and Motivation

Robotic manipulators are increasingly used in domains such as industrial automation, teleoperation, and robot-assisted surgery, where safe interaction with physical environments is essential. Visual feedback provides important workspace information, but it does not directly communicate contact, constraint, or force-regulation cues.

Haptic interfaces address this limitation by enabling bidirectional physical interaction between a user and a virtual or remote environment. In robotic manipulation training, this can help users learn how to regulate force and respond to contact conditions, and prior work has shown that haptic feedback can improve task performance and reduce excessive applied force, particularly for less experienced users [1], [2].

Despite these benefits, access to haptic systems remains limited in research and educational settings. Commercial interfaces provide mature force feedback, but are often expensive and tied to vendor-specific SDKs [1]. There is therefore practical motivation for low-cost, modular haptic platforms suitable for prototyping, experimentation, and robotics training research.

Achieving stable haptic interaction is technically challenging. Real-time haptic rendering commonly targets update rates on the order of 1 kHz with low latency [3], while delay introduces phase lag and reduces maximum achievable virtual stiffness [4]. A practical system therefore requires coordinated mechanical design, embedded control, communication architecture, simulation coupling, and force rendering.

1.2 Problem Definition

This dissertation addresses the design and evaluation of a low-cost real-time haptic interface system for robotic manipulation training. The challenge is to integrate sensing, host communication, simulation, haptic rendering, and actuator command output into a bounded bidirectional loop.

Each stage introduces constraints: actuator capability limits rendered feedback, communication delay limits responsiveness and virtual stiffness, and software architecture determines extensibility.

Existing work demonstrates the value of haptic feedback and the feasibility of affordable haptic devices [5], [6], [7]. However, less emphasis is placed on complete end-to-end architectures where hardware, embedded control, host-side simulation, rendering, and communication are developed and evaluated together. Communication architecture is therefore central, since buffering, scheduling, and transport delay directly affect stability and contact fidelity.

The problem addressed in this project is therefore defined as follows: *to design, implement, and evaluate a low-cost haptic rendering system that is sufficiently modular for experimentation, sufficiently responsive for real-time interaction, and capable of bounded, controllable bidirectional haptic coupling with a simulated robotic environment*.

1.3 Aim and Scope of This Dissertation

The project demonstrates a complete end-to-end haptic architecture covering device construction, embedded firmware, host-side physics simulation, haptic rendering, visualisation, communication testing, timing analysis, and bounded-contact evaluation. The scope is intentionally limited: the dissertation does not attempt calibrated force output, commercial-level rendering fidelity, or a formal user study. Final hardware validation used force feedback on one joint only after actuator failure, so conclusions about full 2-DOF force rendering remain provisional. The principal contribution is therefore system-level design, integration, and evaluation under practical constraints.

1.4 Objectives

To achieve this aim, the project pursued five SMART objectives:

(i) Build a low-cost physical haptic interface with a total hardware cost below £200, capable of measuring joint position and accepting actuator torque commands during closed-loop operation.

(ii) Evaluate whether host–device packet streaming can be sustained at a nominal 1 kHz rate, while separately quantifying packet integrity, burst behaviour, and round-trip latency to determine the effective closed-loop timing limitation.

(iii) Demonstrate that host-side computation is not the dominant timing bottleneck by measuring mean closed-loop host processing latency below 1 ms during integrated operation.

(iv) Verify safety behaviour by checking host-side force-command saturation, embedded torque limiting, torque slew-rate limiting, and watchdog torque removal during communication interruption.

(v) Evaluate physical interaction performance under motor-enabled contact and constrained-motion tests, reporting penetration bounds, rendered-force command consistency, and limitations affecting generalisation to full multi-axis force feedback.

1.5 Contributions of This Work

Prior work on low-cost haptic platforms often focuses on hardware design or isolated demonstrations. This work instead co-develops and evaluates hardware, firmware, communication, simulation, and rendering as one system. Its principal contributions are:

- **A complete end-to-end haptic rendering system.** Custom hardware, embedded firmware, a packet-based communication protocol, and a modular host-side framework with real-time physics and OpenGL visualisation are integrated into a single bidirectional pipeline and evaluated as a unified system. This type of co-design and cross-layer evaluation is less commonly emphasised in existing low-cost academic haptic prototypes.
- **Demonstration of bounded contact interaction under non-negligible communication delay.** Bounded, controllable contact behaviour was observed during the tested interactions despite a measured round-trip latency of approximately 15–16 ms — more than an order of magnitude above the nominal 1 ms local haptic/device-loop period. This behaviour was achieved without a formal passivity guarantee through the combined use of virtual coupling, damping, snapshot-based communication, multi-threaded decoupling, and an embedded torque slew-rate limiter.
- **Quantitative identification of transport delay, not computation, as the dominant performance constraint.** Instrumented runtime logging isolates host-side processing latency from transport-layer delay (as quantified in Section 5.1.3), demonstrating that the communication interface, not the host processor, is the bottleneck. This finding directly motivates communication-architecture improvements as the primary route to higher haptic stiffness in this class of system.

- **Characterisation of USB virtual-COM burst behaviour and a snapshot-based mitigation strategy.** Measured host-side packet inter-arrival times reveal strongly bimodal delivery: most packets arrive in near-simultaneous bursts separated by gaps of 14–16 ms, consistent with buffering and scheduling in the ST-Link virtual-COM, Windows serial-driver, and host communication path. A snapshot-based channel design decouples the haptic rendering loop from this irregular delivery pattern, preventing stale-state accumulation without requiring changes to the communication stack.

2 Literature review

This section reviews the literature needed to justify the system requirements and evaluation approach. It covers haptic interfaces for robotics, rendering and contact methods, stability under delay, communication timing, and low-cost hardware platforms.

2.1 Haptic Interfaces for Robotics

As established in Section 1.1, haptic interfaces provide force feedback that complements visual information and supports fine motor control in robotic manipulation and training contexts. This section reviews the relevant literature on rendering techniques, stability analysis, and low-cost hardware platforms.

Commercial haptic devices such as the 3D Systems *Geomagic Touch* and *Touch X*, the *Force Dimension* product range, and the Haply *Inverse3 X* are widely used in research, training, and simulation contexts [1]. These systems provide kinesthetic force feedback and are typically distributed with proprietary SDK integration workflows, although some also expose lower-level APIs for custom software development.

However, they are generally sold at substantially higher price points than low-cost academic prototypes. For example, the Geomagic Touch and Touch X are publicly listed at approximately US$3,400 and US$13,400 respectively [8], [9], while other systems are commonly distributed via quotation-based sales channels [10], [11]. In contrast, the system developed in this work has a total hardware cost of around £180, highlighting the potential for low-cost systems to provide accessible haptic functionality for research and training applications.

2.2 Haptic Rendering Techniques

Real-time haptic rendering has been extensively studied, with particular focus on contact-based interaction models. Two widely adopted approaches are virtual coupling and the virtual proxy (or "god-object") method introduced by Zilles and Salisbury and later extended by Ruspini *et al.* [3], [12].

In these approaches, the device state is coupled to a constrained proxy within the virtual environment, enabling stable rendering of contact and geometric constraints. Reliable performance typically requires high local update rates on the order of 1 kHz, together with accurate tracking of device state and consistent mapping from Cartesian interaction forces to actuator torques [3]. This requirement applies to the freshness of the local haptic contact model, not only to nominal software loop frequency or packet throughput. In robotic implementations, these mappings are commonly derived using the manipulator Jacobian within operational-space control frameworks [13].

Haptic systems are commonly classified as impedance-type or admittance-type devices, depending on whether the device measures position and outputs force (impedance) or measures force and outputs motion (admittance). Most grounded haptic interfaces, including the system developed in this work, operate as impedance devices due to the relative ease of position sensing and actuation.

Contact detection must also be computationally simple enough for the haptic servo loop. Signed distance fields (SDFs) provide direct access to penetration depth and surface normal information from the sign and gradient of the distance function [14]. Compared with explicit mesh-contact tests or broad-phase bounding-volume methods alone, this representation is well suited to a high-rate proxy-based loop because the contact query produces the quantities required for both proxy projection and normal-force calculation.

The literature also motivates separating high-rate local haptic contact handling from slower application and graphical layers. Ruspini *et al.* describe a multilevel haptic rendering framework for complex graphical environments, while constraint-based proxy methods maintain a local contact point or proxy state that is updated continuously subject to contact constraints [3], [12], [15]. This supports the architectural distinction between a high-frequency local contact model and slower subsystems responsible for broader scene definition, visualisation, and application-level updates.

2.3 Stability in Haptic Systems

Stable haptic interaction imposes strict requirements on latency, control architecture, and numerical implementation. Adams and Hannaford demonstrated that virtual coupling can provide stability guarantees by decoupling device dynamics from the virtual environment under passive interaction assumptions [4].

More generally, stability in haptic systems is often analysed using passivity theory. Colgate and Schenkel showed that discrete-time haptic interfaces can become unstable when energy is artificially generated within the feedback loop due to sampling, delay, or numerical approximation errors [16]. Their passivity framework establishes conditions under which a haptic system remains energetically passive, thereby ensuring stable interaction with arbitrary passive users and environments.

These results highlight the sensitivity of haptic systems to delay and discrete-time effects. In practical implementations, communication latency and asynchronous computation can result in effective energy injection into the feedback loop, reducing stability margins and limiting the achievable virtual stiffness. Passivity-based analyses therefore motivate treating loop delay as a direct limit on renderable stiffness: as delay increases, the range of stable virtual coupling parameters becomes more restricted. This is particularly relevant in systems where communication delay is non-negligible relative to the control timestep, motivating the use of control structures that explicitly manage dynamic coupling, damping, and energy injection in order to maintain stable interaction.

2.4 Communication Timing in Real-Time Haptic Loops

Haptic rendering places unusually strict requirements on communication timing because the device state and returned force command form part of the same feedback loop. Prior work on sampled-data haptic interaction shows that delay and discretisation reduce stability margins [4], [16], while USB latency studies show that USB-based real-time control links can introduce delays on the order of milliseconds [17]. Interfaces that are convenient for prototyping, such as USB virtual COM ports using the USB CDC class, can therefore become a limiting factor when packets are buffered by the interface firmware, host driver, or operating system scheduler. Even when the embedded controller transmits samples at a regular rate, non-uniform host-side delivery can increase effective feedback delay if the control loop processes stale queued states.

For this reason, communication design should be considered alongside control design in practical

haptic systems. A latest-value, or snapshot-based, transfer pattern is one suitable architectural response: the haptic loop consumes the newest available state rather than draining a backlog of historical samples. This does not remove transport delay, but it can prevent buffered packets from accumulating into additional application-level latency.

2.5 Low-Cost Haptic Hardware

Advances in low-cost components, including 3D-printed structures, microcontrollers, and magnetic encoders, have enabled the development of accessible haptic devices using widely available hardware. This has led to increasing interest in open and modular platforms for research and education [1].

The *Hapkit* platform developed by Martinez *et al.* demonstrates that functional haptic devices can be realised at very low cost (approximately US$50) using 3D-printed components [5]. Subsequent work extended this approach to modular multi-degree-of-freedom configurations, including both pantograph and serial mechanisms [6].

Similarly, Lavatelli *et al.* presented an open-source low-cost 2-DOF haptic device, demonstrating that multi-degree-of-freedom actuation can be achieved within an academic budget while maintaining useful interaction capability [7].

However, much of this work focuses primarily on hardware design or educational platforms. Comparatively less emphasis is placed on fully integrated systems that combine low-cost hardware with real-time communication, simulation coupling, and extensible software architectures.

2.6 Summary of Literature

The literature establishes that haptic feedback improves performance in robotic manipulation tasks and provides well-developed methods for stable force rendering, particularly through virtual coupling and passivity-based control frameworks. Low-cost hardware platforms have also demonstrated that accessible haptic devices are feasible.

However, there remains a specific gap between simplified hardware demonstrations and fully integrated real-time systems in which communication architecture, stability strategy, simulation cou-

pling, and force rendering are evaluated together. This gap directly motivates the co-designed system architecture presented in Section 3.

3 System Design and Implementation

This section describes how the haptic device, embedded firmware, communication layer, and host software were developed into the final integrated prototype. The system prioritises bounded real-time interaction, modularity, and reproducibility within the constraints of a low-cost build, with time-critical embedded control separated from asynchronous host-side simulation, rendering, and logging.

3.1 System Overview

The proposed system implements a bidirectional haptic interaction loop consisting of four principal layers: the haptic device hardware, embedded control firmware, host-side simulation and haptic rendering, and graphical visualisation. A conceptual overview of the signal and force flow is shown in Fig. 1.

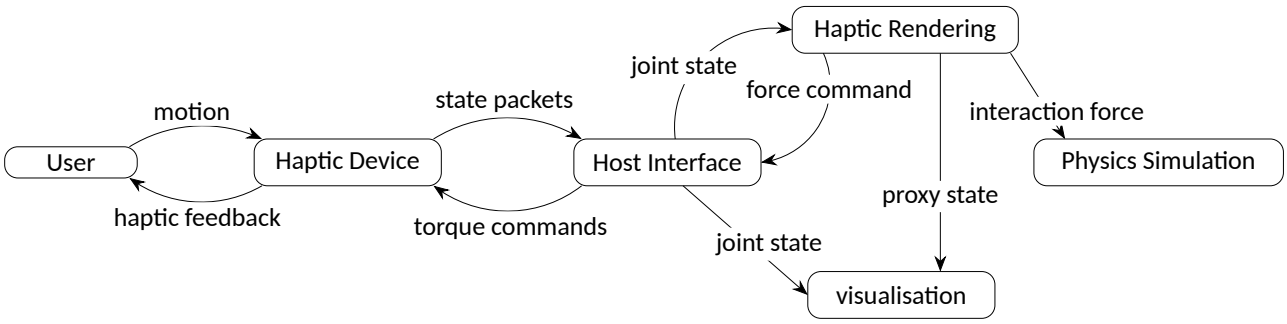


Fig. 1. High-level architecture of the proposed haptic system. User motion is measured by the haptic device and transmitted to the host interface. The host distributes state information to the haptic rendering and visualisation subsystems, while the haptic rendering module computes interaction forces, updates the simulation, and generates feedback commands that are returned to the device as actuator torque commands.

Time-critical control tasks are executed on the embedded system at high frequency, while simulation, rendering, and visualisation are performed asynchronously on the host.

3.2 Mechanical and Embedded Platform

The physical interface is a planar two degree-of-freedom serial mechanism designed for low-cost fabrication and straightforward maintenance. A two-link layout was selected to provide a usable workspace while keeping actuator count, wiring complexity, and control overhead manageable.

The hardware was developed iteratively rather than as a single final build. The first design established the planar two-link geometry, approximate 150 mm link lengths, and encoder-on-joint measurement strategy. Integration work then exposed three constraints that shaped the final device: low-friction backdrivable joints, more base-joint torque than direct drive could provide, and a motor-control route that could be debugged reliably within the project timescale.

These constraints led to the final mechanical arrangement shown in Fig. 2 and Appendix E. The structure is primarily 3D printed, with each joint supported by steel shafts and rolling element bearings to reduce friction and improve backdrivability. Actuation is provided by brushless gimbal motors (T-Motor GB2208)[18]. The base joint uses a 3:1 GT2 belt reduction stage to increase available output torque, while the elbow joint remains direct-drive to minimise inertia, backlash, and assembly complexity.

An adjustable idler pulley is integrated into the belt stage to enable tensioning and reduce backlash. Despite this, non-ideal behaviour such as compliance and occasional slip under high load was observed, as discussed in Section 6.4.

To ensure accurate joint angle measurement, absolute magnetic encoders (AS5048)[19] are mounted directly on the joint axes and interfaced via SPI. This is particularly important for the belt-driven joint, where transmission compliance would otherwise introduce discrepancies between motor-side and output angles.

For the belt-driven actuator, an additional AS5600 magnetic encoder[20] is mounted on the motor shaft and interfaced via I^2C. This provides rotor position feedback required for field-oriented control (FOC), while the joint encoder provides accurate output angle measurement. A practical development challenge was positioning the rear motor-shaft encoder close enough to the magnet for reliable readings while still leaving a rigid fastening path between the motor and printed body; this affected the motor mount geometry and required the encoder support to be integrated into the structural bracket rather than added as a separate part.

Motor-driver development also changed during the project. The initial approach targeted STM32

B-G431-ESC1 drivers because they offered integrated power electronics and current sensing for more quantitative torque control. In practice, combining these drivers with the selected encoder arrangement proved unreliable, so the design pivoted to SimpleFOC Mini driver modules controlled from the Nucleo board. This reduced force-control fidelity because phase current was no longer measured directly, but produced a more robust embedded platform and preserved the main integration, communication, and safety objectives.

A complete mechanical assembly, including component layout and link geometry, is provided in Appendix E. The assembled prototype is shown in Fig. 2, illustrating the realised mechanical structure and actuator configuration.

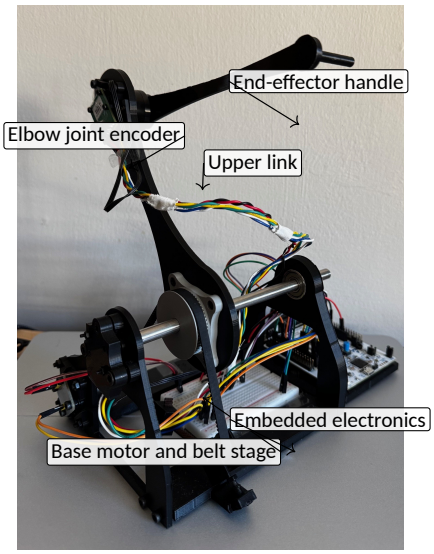


Fig. 2. Final assembled 2-DOF haptic interface prototype, annotated with the main mechanical and embedded components.

Embedded control runs on an STM32 Nucleo-G431RB[21] platform with SimpleFOC Mini drivers and the SimpleFOC software stack [22]. Both actuators operate in torque-mode FOC [23]. A communication watchdog is implemented in firmware: if valid host commands are not received within the timeout window, output torques are forced to zero. Additionally, a torque slew-rate limiter is applied to each actuator command, constraining the rate of change of commanded torque to reduce high-frequency oscillations during rapid contact transitions.

Due to hardware constraints, no direct force sensing or current measurement was implemented; as a result, torque output is estimated rather than directly measured. Additionally, one actuator failed during development, meaning final hardware validation was conducted using single-axis actuation, although both joints remained available for sensing.

3.3 Communication Protocol and Device Interface

Host–device communication uses fixed-format binary packets over a USB virtual serial link at a nominal 1 kHz packet streaming rate. Device state packets contain a sequence number, MCU timestamp, and joint angles; torque command packets contain the command sequence, referenced state sequence, and commanded joint torques. Sequence numbers support drop and ordering checks, while timestamps support timing analysis. Both packet types include a two-byte header for stream re-synchronisation and a checksum for integrity checks. The target packet rate defines the intended communication cadence, but not the end-to-end freshness of the closed-loop haptic state.

On the host, serial I/O and protocol handling are split across two layers: *SerialLink* providing a low-level COM read/write abstraction, and *DeviceAdapter* handling packet parsing, forward kinematics, Cartesian force-to-joint torque mapping, and message-bus integration.

The device adapter runs in a dedicated nominal 1 kHz loop. At each cycle, it retains the newest valid state packet, publishes tool pose and velocity to the haptic engine, consumes the latest wrench command, computes joint torques using the planar Jacobian transpose relation $\tau = J^T F$, and transmits the resulting torque command packet.

This keeps control decisions based on the newest state available to the host and avoids queueing stale samples in the device path.

3.4 Host Software Architecture

The host application is implemented as a modular multi-threaded C++ system, split into the renderer, world manager, physics engine, haptic engine, and device adapter. This separates user-interface rendering, scene ownership, physics stepping, haptic force computation, and serial device handling, rather than driving the whole application from a single monolithic update loop.

Inter-thread communication uses a message bus with channel types chosen to match the data being transferred. Queue channels preserve ordered commands and events, such as world edits and wrench messages. Snapshot channels store only the newest value for continuously updated state, such as tool pose and world snapshots, so high-rate loops consume current data without building a backlog of obsolete samples.

Subsystems exchange complete message objects rather than shared mutable structures. The world

manager owns scene mutation, the device adapter owns serial parsing and torque-packet construction, and the haptic engine consumes snapshots and publishes force commands. These ownership boundaries make the thread responsibilities explicit and reduce cross-subsystem coupling during integration.

3.4.1 World and Rendering Subsystems

WorldManager is the authoritative owner of virtual scene state. Objects store identity, geometry reference, pose, appearance, role labels, and physics properties. Subsystems do not mutate world data directly; instead, they publish commands which are applied centrally by the world manager.

Rendering is handled by an OpenGL subsystem running on the main thread. Each frame reads the latest world snapshot, resolves geometry and mesh handles, composes model transforms from object pose, and renders via shared shader and mesh abstractions. An ImGui interface provides runtime controls for object editing and parameter tuning; UI edits are emitted as world commands to preserve ownership boundaries.

3.4.2 Haptic Rendering Engine

The haptic engine executes at a nominal 1 kHz in a dedicated loop. At each cycle, it reads the latest world snapshot and newest tool state, performs contact queries, computes force output, and publishes results to device, physics, and visualisation channels. This rate refers to the local haptic rendering computation cadence, with end-to-end freshness limited by communication and asynchronous simulation timing.

Contact detection is performed using signed distance field (SDF) queries, with penetration depth and surface normals obtained from the local distance field gradient [14]. For each candidate object, the tool position is transformed into object-local coordinates and queried against that object's SDF. If penetration is detected (negative signed distance), the tool point is projected to the surface to define the proxy position; otherwise, the proxy follows the tool in free space.

Force output is generated using a proxy-based virtual coupling model, in which a spring–damper relationship is defined between the device position and a contact-constrained proxy [4]:

$$\mathbf{F} = K\left(\mathbf{x}_p - \mathbf{x}_t\right) + D\left(\mathbf{v}_p - \mathbf{v}_t\right)$$

where $\mathbf{x}_p, \mathbf{x}_t$ are proxy and tool positions and $\mathbf{v}_p, \mathbf{v}_t$ are corresponding velocities. In the implemented configuration, $K = 500\,\mathrm{N\,m^{-1}}$, $M = 0.02\,\mathrm{kg}$, and $D = 0.7 \cdot 2\sqrt{KM}$. The stiffness was selected as a conservative value for the measured communication delay while still producing clearly perceptible contact feedback; Section 5.2.2 checks command-side model consistency, while Section 6.3 discusses the resulting contact behaviour.

This follows the standard virtual coupling formulation, although the present implementation does not explicitly enforce passivity-based design constraints. In particular, the proxy state is recomputed directly from the current tool position and contact geometry at each timestep, rather than being evolved as a persistent constrained dynamic state. As a result, the implementation captures the force-generation behaviour of virtual coupling but does not realise the full passivity-based formulation described by Adams and Hannaford, nor the continuously evolved contact proxy used in constraint-based haptic rendering approaches [3], [12], [15].

Force commands are therefore generated using a conservative coupling configuration, with damping and actuator-side limiting used to reduce abrupt changes during contact transitions.

To maintain safe interaction, the rendered-force command magnitude is clamped to a maximum of 31 N before publication.

3.4.3 Physics Simulation Integration

Rigid-body dynamics are simulated using NVIDIA PhysX. The physics subsystem runs on its own simulation thread and consumes wrench messages from the haptic loop. Incoming forces are converted to impulses $\mathbf{J} = \mathbf{F}\,\Delta t$ over the effective timestep and applied at the contact point to the corresponding actor.

A fixed-step accumulator scheme is used to decouple simulation stability from variable external loop timing. When world topology or physics properties change, actors are rebuilt from the latest authoritative world snapshot. After each simulation update, dynamic poses are written back to the world manager so subsequent rendering and haptic queries operate on consistent state. In the realised architecture, the physics engine therefore provides the authoritative global scene evolution at its own rate, while the haptic engine consumes snapshots of that state rather than independently evolving a local dynamic model of contacted objects.

3.5 Logging and Communication Validation

To avoid introducing disk latency into the 1 kHz control path, runtime logging is performed asynchronously. The device thread publishes structured timing and state messages to dedicated logging channels, and a background logger thread drains these channels into memory.

At shutdown, logs are exported to `device_timing.csv` for matched state-command timing records and `device_state_log.csv` for every valid parsed state packet. The timing log captures the packet parsing, tool publication, wrench consumption, and torque transmission timestamps used in Section 5.1.3.

A separate `serial_tests` executable characterises serial performance independently of the full application. It supports packet rate/integrity testing and ping–echo round-trip latency testing, using the same buffered parser, header search, checksum validation, and re-synchronisation approach as the main device interface.

4 Experimental Methodology

This section defines the test procedures used to evaluate communication performance, closed-loop timing behaviour, simulation fidelity, safety behaviour, and hardware interaction characteristics of the proposed haptic rendering system. The section focuses on what was tested and how measurements were collected; quantitative outcomes are reported in Section 5 and interpreted in Section 6.

4.1 Communication Performance Tests

Communication performance was characterised using the dedicated `serial_tests` utility described in Section 3.5. Its rate/integrity and ping–echo modes export CSV files for offline statistical analysis.

Tests were conducted across target transmission rates from 100 Hz to 2000 Hz to assess the effect of increasing packet frequency on reliability and timing performance.

4.1.1 Packet Integrity and Rate Tests

Packet integrity and communication timing were evaluated using the rate-test mode of the communication test utility.

For each target transmission frequency, the firmware was configured to stream fixed-format device state packets at a constant rate. Tests were performed at nominal frequencies of 100 Hz, 250 Hz, 500 Hz, 1000 Hz, 1250 Hz, and 2000 Hz.

For each test condition, the host continuously received and parsed incoming packets over the USB virtual COM interface. The recorded performance metrics included valid packet count, dropped packets inferred from sequence gaps, out-of-order packets, host-observed inter-arrival time, effective receive rate, and inter-arrival jitter.

4.1.2 Round-Trip Latency Test

Round-trip latency was evaluated using a ping–echo communication test.

In this test, the host transmitted timestamped ping packets to the embedded controller, which immediately returned an echo packet containing the original sequence number and timestamp.

Tests were performed using transmission intervals of 1 ms, 2 ms, and 10 ms. For each test, the evaluated metrics included mean, median, 95th percentile, and 99th percentile RTT, along with minimum and maximum RTT values and timeout counts.

These results were used to characterise communication latency and timing consistency under different transmission loads.

4.1.3 End-to-End Host-Side Timing Test

In addition to the standalone communication tests, end-to-end timing of the host-side haptic pipeline was evaluated using the runtime logging framework described in Section 3.5. Logs captured during normal closed-loop operation were analysed in MATLAB to compute host-side latency, intermediate pipeline delays, matched state-command consistency, parsed state-packet inter-arrival timing, sequence gaps, parsed packet rate, and MCU-side timing trends.

Because the host and microcontroller clocks were not synchronised, the analysis was restricted to host-side timing differences and trend-based interpretation of MCU timestamps. Absolute one-way latency between host and device could therefore not be inferred directly.

4.2 Simulation Validation

Simulation validation tests were designed to assess the command-side behaviour of the haptic rendering model independently of physical actuator dynamics.

The simulation tests focused on planar virtual-wall behaviour. A single-depth hold test assessed whether the rendered force command remained steady during sustained penetration. A multi-depth hold test quantified the command–penetration relationship by holding the end-effector at several penetration depths and fitting the logged rendered-force command data. Additional free-space and deep-penetration checks verified that the renderer produced negligible force command outside contact and that host-side force-command saturation was applied under extreme penetration. Physical closed-loop interaction with actuator feedback was evaluated separately through the repeated-contact and constrained-motion hardware tests in Sections 4.4.1 and 4.4.2.

4.2.1 Virtual Wall Test

A virtual wall test validated unilateral contact behaviour by moving the end-effector toward and against a planar boundary. The evaluation examined whether force was near zero outside contact, remained steady during a single-depth hold, increased approximately linearly with penetration depth, and matched the implemented spring–damper coupling model while the proxy remained constrained to the wall.

4.2.2 Free-Space and Saturation Checks

Free-space and deep-penetration checks were used to verify the boundary cases of the renderer. The free-space check confirmed that no significant force was generated outside contact. The deep-penetration check verified that large virtual penetrations were limited by the configured host-side force clamp before torque commands were generated.

4.3 Safety Validation

Safety validation tests evaluated whether the system remained bounded and behaved predictably under both normal and fault conditions. These tests focused on checking torque saturation, rate limiting, and watchdog-based fail-safe behaviour. The reported safety metrics were saturation fraction, maximum raw and clamped torque, applied torque cap, measured torque slew-rate percentiles, watchdog activation count, total watchdog-active time, activation duration, and sequence-matched command consistency.

4.3.1 Torque Saturation Test

A torque saturation test verified that host-side force limits and embedded torque clamps kept actuator commands within safe bounds during high-force contact.

4.3.2 Rate Limiting Test

A rate limiting test evaluated the effect of constraining the rate of change of commanded torque at the embedded controller level.

Testing verified that the embedded slew-rate limiter smoothed rapid command changes into bounded torque ramps and reduced high-frequency oscillation during contact transitions.

4.3.3 Watchdog Timeout Test

A watchdog timeout test verified safe system behaviour under communication loss by interrupting the incoming command stream during operation.

The test verified that the watchdog triggered when command updates ceased, removed commanded torque within the timeout window, and returned the device to a zero-torque command state.

4.4 Hardware Validation

Hardware validation assessed the behaviour of the physical actuation system and overall device interaction using the updated runtime logging framework. Because no calibrated force sensor or

phase-current measurement was available, the tests did not measure physical output force directly. Instead, logged position, proxy position, rendered force command, host torque command, and device-side applied torque were used as quantitative indicators of contact behaviour, bounded motion, command consistency, and practical interaction quality.

4.4.1 Repeated Contact Test

A repeated-contact test evaluated the behaviour of the full closed-loop system during sustained interaction with a virtual surface.

The end-effector was brought into contact with a virtual object while actuator feedback was enabled. Logged data were used to assess whether penetration remained bounded and whether the commanded force–penetration relationship remained consistent with the implemented virtual coupling model. The reported metrics were mean, percentile, and maximum penetration depth, together with the fitted command–penetration stiffness and coefficient of determination. Qualitative observations of oscillation, chatter, and usability were used only to contextualise the logged results.

4.4.2 Constrained Motion Hardware Test

Hardware constrained-motion behaviour was evaluated during physical operation of the device. The end-effector was brought into contact with a planar virtual surface and moved along it while maintaining contact. Quantitative metrics were computed over a sustained-contact window, excluding entry and release transients. The evaluation assessed whether the logged force command was predominantly normal to the surface, whether normal displacement remained bounded, and whether tangential motion could be produced without large oscillatory excursions. The reported metrics were mean and maximum penetration, normal-to-tangential force-command ratio, sustained-contact logged force-command magnitude, and force-command coefficient of variation.

5 Results

This section reports the measured outcomes of the Section 4 tests. It presents communication timing, simulation behaviour, safety outcomes, and hardware interaction observations. Interpretation of these findings and their design implications is deferred to Section 6.

5.1 Communication Performance

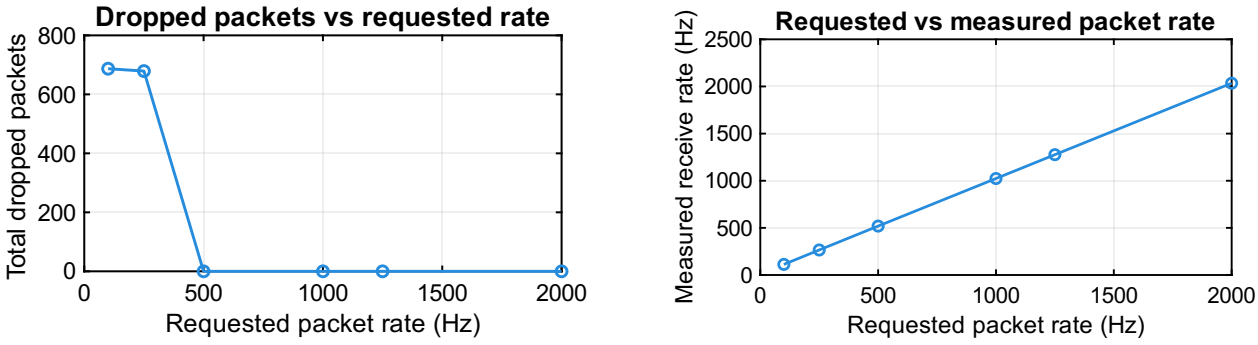
Communication performance was evaluated using standalone serial tests and runtime logging of the integrated haptic application.

5.1.1 Packet Rate and Integrity Results

Packet rate testing showed that communication performance remained consistent over a wide range of requested transmission frequencies, although the behaviour was not uniform across all tested rates.

At lower requested rates of 100 Hz and 250 Hz, packet loss was observed, with drop fractions of approximately 20.3% and 8.5% respectively. In contrast, for requested rates of 500 Hz and above, no dropped packets were observed in the recorded data, as shown in Fig. 3(a), where packet loss occurs only at the lowest transmission rates. Out-of-order packets were extremely rare across all higher-rate tests, with only a single out-of-order event recorded in each case.

The measured receive rate tracked the requested transmission rate closely across the full test range. As shown in Fig. 3(b), the relationship between requested and measured rate was approximately linear, indicating that the communication system maintained throughput proportional to the commanded transmission frequency.



(a) Dropped packets

(b) Rate tracking

Fig. 3. Communication performance across transmission rates. Packet loss occurred only at low frequencies, while measured receive rate tracked the requested rate approximately linearly.

Inter-arrival jitter, measured as the standard deviation of packet inter-arrival time, was present in all tests, with standard deviation decreasing as the requested rate increased, as shown in Fig. 4. However, percentile analysis revealed that occasional large timing excursions persisted even at

higher rates, indicating intermittent latency spikes. While average jitter decreased with increasing rate, worst-case timing excursions remained present.

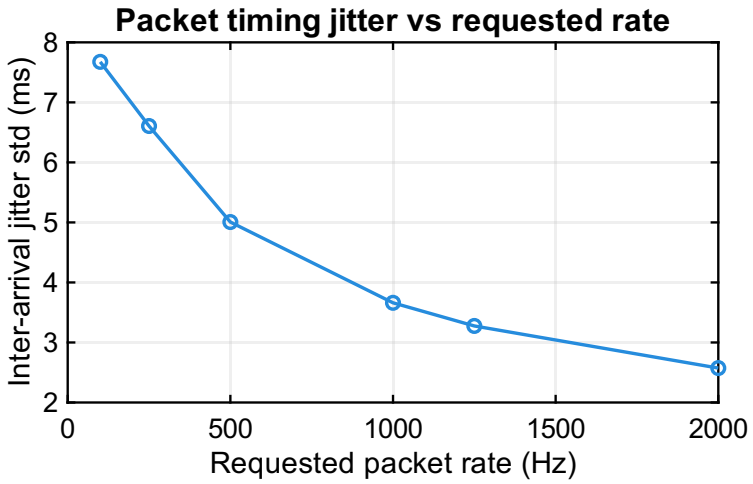


Fig. 4. Packet inter-arrival jitter decreases with requested rate, although occasional large timing excursions remain present.

Overall, the serial link supported nominal packet throughput at or above 1 kHz with negligible packet loss in the tested configuration. This result describes packet generation and throughput, rather than low-latency delivery or 1 ms closed-loop state freshness.

5.1.2 Round-Trip Latency Results

Round-trip latency results were highly consistent across all tested ping intervals. For nominal transmission intervals of 1 ms, 2 ms, and 10 ms, the success rate was 100% in all cases, with no timeouts observed over 1000 samples per test. The mean measured round-trip latency remained approximately constant across all three test conditions, with values of 15.60 ms, 15.64 ms, and 15.67 ms respectively. Similarly, the 95th and 99th percentile RTT values were tightly grouped around 16.2–16.4 ms, indicating consistent and predictable timing behaviour across repeated measurements. This behaviour is further illustrated in Fig. 5, where the cumulative distributions for all three tests overlap closely and rise sharply at approximately 15–16 ms, indicating that RTT is largely independent of the selected ping interval.

The standard deviation of RTT remained below 0.52 ms in all cases, suggesting relatively low variability in round-trip timing. Minimum and maximum observed RTT values showed occasional larger deviations from the typical latency range, particularly in the 1 ms test, but these did not significantly affect the overall distribution.

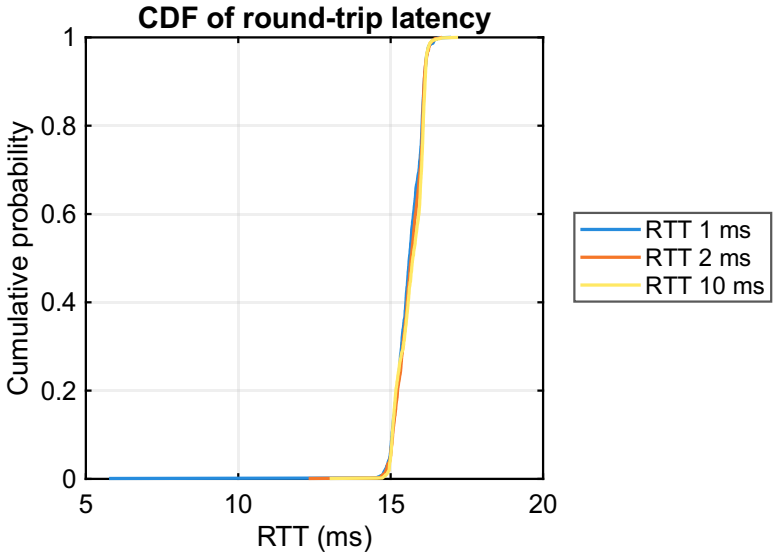


Fig. 5. Cumulative distributions of RTT for 1 ms, 2 ms, and 10 ms ping intervals. The overlapping curves indicate a largely fixed 15–16 ms delay component.

These results indicate that RTT was dominated by a largely fixed delay component and changed little with ping interval. The measured delay remained predictable due to low variability and zero observed timeouts.

5.1.3 End-to-End Host-Side Timing Results

End-to-end timing behaviour of the integrated haptic pipeline was evaluated using runtime logging of matched state-command cycles and raw parsed state packets.

Analysis of `device_timing.csv` showed that all recorded control cycles were successfully matched, with a matched fraction of 1.000, indicating consistent correspondence between received state packets and transmitted torque commands.

The host-side end-to-end latency, defined as the time between state-packet parsing and completion of torque command transmission, was found to be low. The mean latency was 0.299 ms, with a median of 0.104 ms. The 95th percentile latency was 0.546 ms, and the 99th percentile was 2.04 ms, with a maximum observed latency of 3.35 ms. As shown in Fig. 6, the distribution was concentrated at low values, with prominent peaks near 0 ms and around 0.5 ms, and only a limited number of higher-latency events.

Breakdown of intermediate pipeline delays showed that the majority of this latency was associated with serial transmission. The mean transmission duration was 0.249 ms, while the delay associated with publishing tool state and processing the haptic pipeline remained small, with mean values

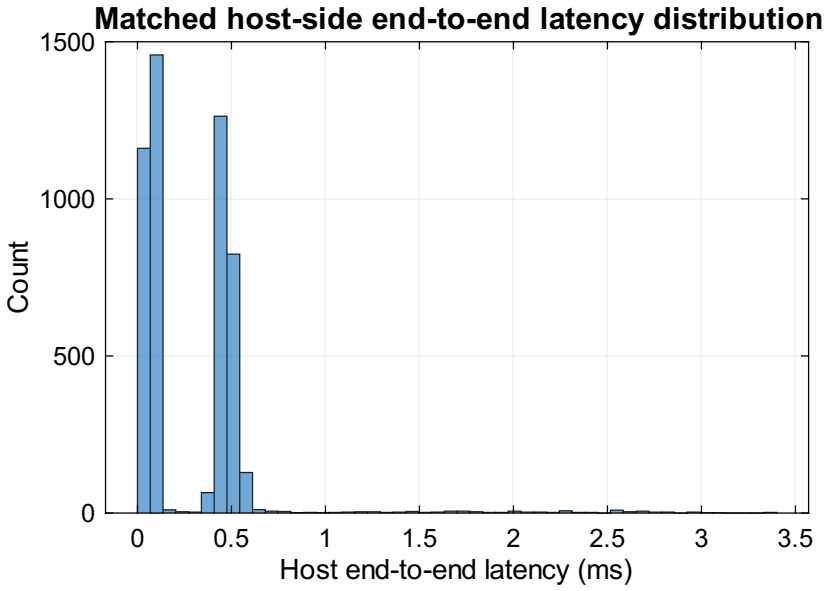


Fig. 6. Matched host-side latency from state-packet parsing to torque-command transmission, concentrated at low values with few higher-latency outliers.

of 0.024 ms and 0.0044 ms respectively. This indicates that host-side computation and message-passing overhead were negligible relative to communication time.

Sequence analysis showed that command sequence progression remained consistent, with a mean command sequence gap of 1.14 and a maximum of 6, indicating that command updates were generally applied sequentially with minimal skipping. In contrast, the matched state sequence gap was significantly larger, with a mean of 33.45. This reflects the fact that the control loop operated on a subset of the available incoming state packets rather than processing every parsed packet. Here, sequence gap denotes the difference between consecutive sequence identifiers, such that a value of 1 corresponds to consecutive packets or commands and larger values indicate skipped intermediate sequence numbers.

A total of 54 161 valid state packets were parsed during the test, compared to 5 048 matched control cycles, confirming that the control loop operated on a subset of the incoming packet stream rather than consuming every parsed packet. The estimated parsed packet rate, computed from the mean host-side inter-arrival time, was approximately 1006 Hz, which is close to the expected 1 kHz transmission rate.

Analysis of host-side parsed-packet inter-arrival timing revealed significant variability. The mean inter-arrival time was 0.994 ms, but the median was only 0.0016 ms, reflecting that many packets were processed in immediate succession within bursts. The large difference between mean and median inter-arrival time indicates that packets were frequently processed in rapid bursts follow-

ing periods of delay, rather than arriving at uniform intervals. This behaviour is illustrated in Fig. 7, where a dominant peak near 0 ms is accompanied by a much smaller secondary concentration around 14–16 ms. The 95th and 99th percentile inter-arrival times were 14.5 ms and 15.8 ms respectively, confirming the presence of intermittent host-side stalls.

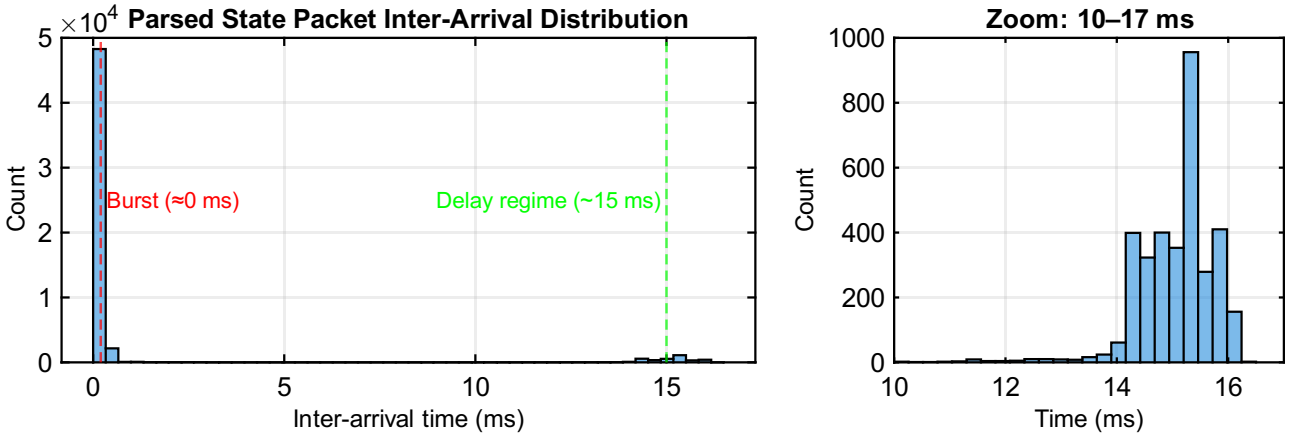


Fig. 7. Host-side parsed-packet inter-arrival distribution. The secondary peak corresponds to the 95th–99th percentile range of 14.5–15.8 ms.

Despite this non-uniform host-side processing pattern, analysis of MCU-side timestamps showed that the embedded system generated state packets at a consistent rate. The effective MCU period per state was approximately 1.00 ms, corresponding to a nominal packet generation rate of 1000 Hz, with very low variability.

Taken together, the embedded controller generated packets at a regular rate, while host-side packet timing was non-uniform. Host-side processing latency nevertheless remained low. These results separate local packet generation and host computation from the effective age of the state used for haptic feedback.

5.2 Simulation Validation Results

Simulation validation was performed as described in Section 4.2, with actuator output disabled. These tests evaluated the haptic rendering model independently of physical actuator dynamics.

5.2.1 Virtual Wall Contact Response

The virtual wall response was evaluated using a single-depth hold test. During contact, the logged rendered-force command magnitude had a mean value of 11.46 N and a standard deviation of 0.30 N, corresponding to a coefficient of variation of 0.026.

As shown in Fig. 8, the rendered-force command was generated only when the end-effector penetrated the virtual wall. When the end-effector was outside the wall, the command-side force output remained approximately zero.

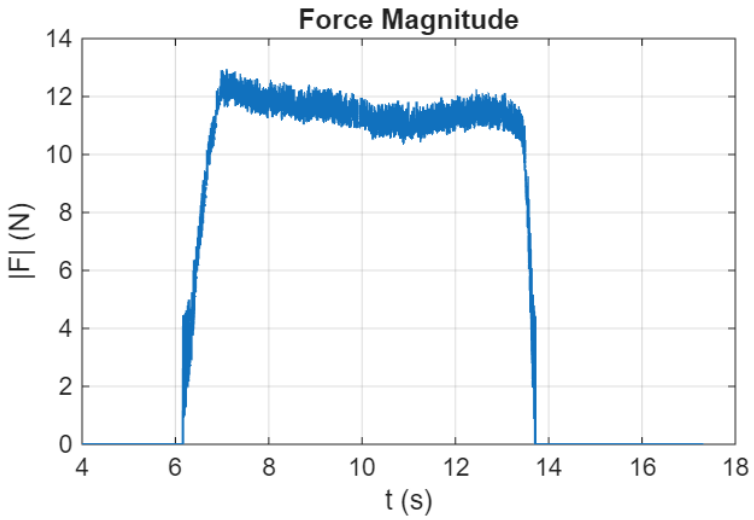


Fig. 8. Single-depth virtual wall contact test. The rendered $|\mathbf{F}|$ remains steady during sustained contact, with mean 11.46 N, standard deviation 0.30 N, and coefficient of variation 0.026.

5.2.2 Force–Penetration Characterisation

A multi-depth hold test was used to check the command-side relationship between virtual penetration depth and logged rendered-force command magnitude. A linear fit to the command–penetration data produced an effective stiffness of 489.57 N m^{-1} with $R^2 = 0.9973$, as shown in Fig. 9. This is an internal model-consistency check, not a measurement of physical output force.

This closely matches the configured virtual coupling stiffness of 500 N m^{-1} over the tested range.

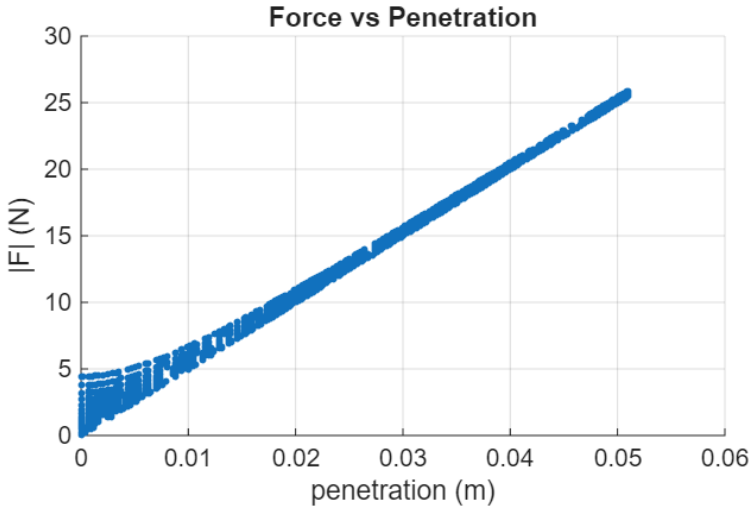


Fig. 9. Force–penetration relationship from the multi-depth hold test, with fitted stiffness 489.57 N m^{-1} and $R^2 = 0.9973$.

5.2.3 Free-Space and Saturation Behaviour

A no-contact test verified that the haptic engine produced no significant force output in free space.

A deep-penetration test was also performed to verify host-side force limiting. Under large penetration, the commanded force saturated at approximately 31 N, as shown in Fig. 10.

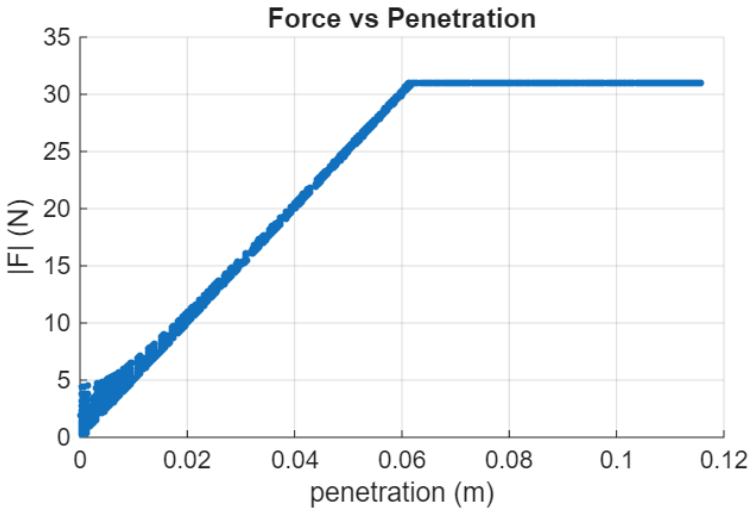


Fig. 10. Deep-penetration simulation test showing force-command saturation at the host-side limit of approximately 31 N.

5.3 Safety Validation Results

Safety validation evaluated whether torque commands remained bounded during high-force contact, rapid command transitions, and communication loss.

5.3.1 Torque Saturation

Torque saturation was evaluated using the deep-penetration test. Host-side saturation occurred in 31.10% of samples, with the maximum raw torque command reaching 9.35 N m before being clamped to 7.66 N m. On the device side, the maximum applied torque was limited to 6.50 N m, matching the configured firmware torque cap.

5.3.2 Rate Limiting

The embedded torque slew-rate limiter was evaluated from the applied torque logs. The measured 95th percentile slew rate was 55.000 N m s^{-1}, the 99th percentile was 55.005 N m s^{-1}, and the

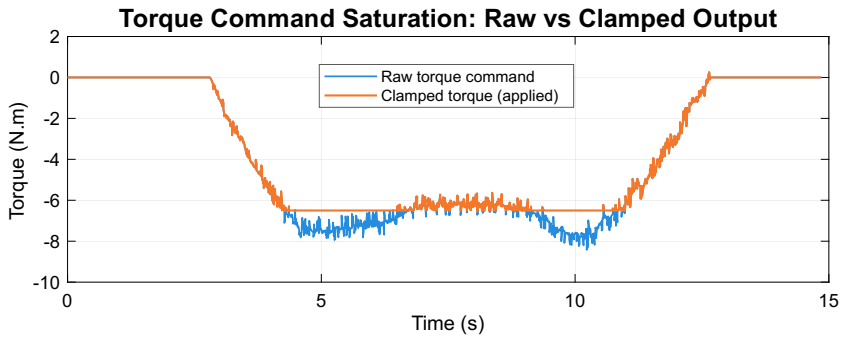


Fig. 11. Torque saturation during deep-penetration testing: raw host commands are clamped before transmission and applied torque remains bounded by the 6.50 N m firmware cap.

maximum measured slew rate was 55.010 N m s^{-1}. These values closely match the configured firmware limit of 55 N m s^{-1}.

The slew-rate distribution is shown in Fig. 12. The dominant peak at 0 N m s^{-1} corresponds to steady-state operation, while the sharp upper bound occurs at the configured 55 N m s^{-1} limit.

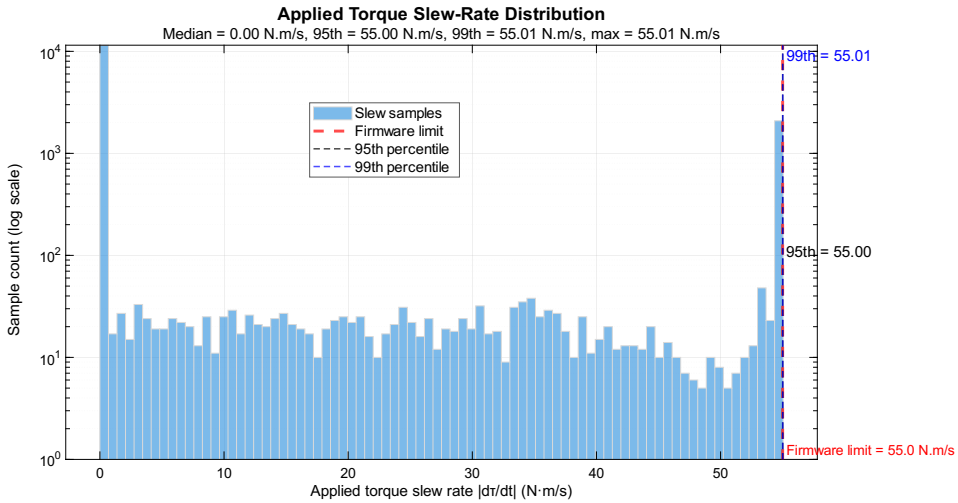


Fig. 12. Applied torque slew-rate histogram showing a sharp upper bound at the firmware limit of 55 N m s^{-1}.

5.3.3 Watchdog Timeout Behaviour

The watchdog was evaluated by deliberately interrupting host torque-command transmission. A total of 259 watchdog activation events were recorded, corresponding to 6.81% of device samples. The total watchdog-active time was 0.529 s, with a mean activation duration of 0.0020 s and a maximum duration of 0.311 s.

As shown in Fig. 13, applied torque was removed during watchdog activation.

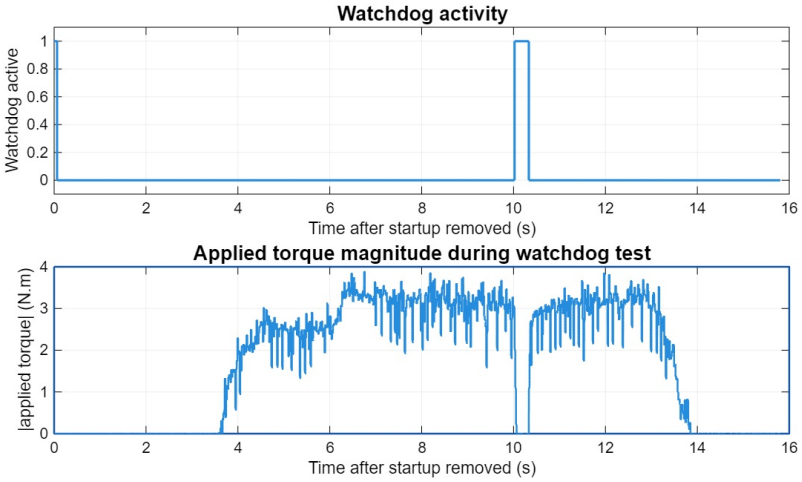


Fig. 13. Watchdog validation during interrupted host command transmission. Applied torque is removed when the watchdog becomes active.

5.3.4 Command Consistency

Sequence-matched host and device logs showed a mean applied torque magnitude error of 0.134 N m between host commands and device-side applied torque. For typical commanded torques of approximately 2–4 N m, this corresponds to roughly 3–7% error. Larger differences occurred during transient command changes, especially when the slew-rate limiter and safety clamps were active.

5.4 Hardware Validation Results

Hardware validation was performed with actuator feedback enabled. Because direct force sensing and current measurement were not available, these results quantify interaction behaviour using logged position, force-command, and torque-command data rather than calibrated physical force output. Final force-feedback testing was limited to single-axis actuation due to actuator failure, although both joints remained available for position sensing.

5.4.1 Repeated Contact With Virtual Wall

During repeated motor-enabled contact with the virtual wall, penetration remained bounded. The mean penetration depth was 0.0342 m, the 95th percentile penetration depth was 0.0498 m, and the maximum penetration depth was 0.0517 m.

The command–penetration relationship during this test remained approximately linear, with an effective command-side stiffness of 490.99 N m^{-1} and $R^2 = 0.9979$. This indicates that the rendered-

force command remained internally consistent with the virtual coupling model during closed-loop interaction. The comparison with the simulation-only value of 489.57 N m^{-1} in Section 5.2.2 is based on logged command data rather than direct force measurement.

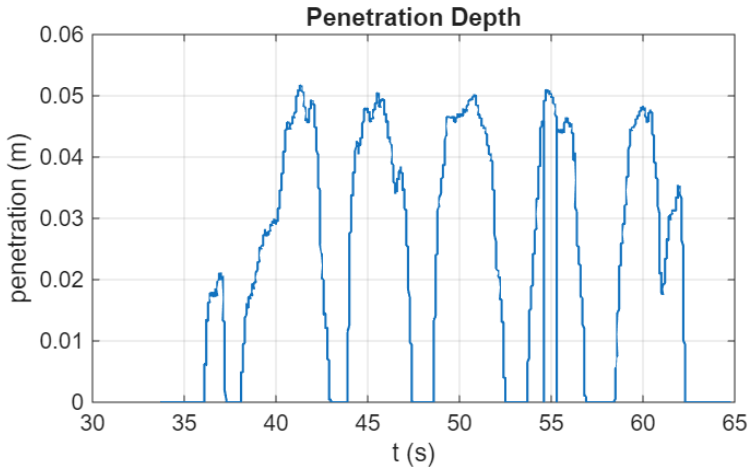


Fig. 14. Motor-enabled repeated contact test. Maximum penetration was approximately 0.052 m; the command–penetration fit gave 490.99 N m^{-1} with $R^2 = 0.9979$.

5.4.2 Constrained Motion Along a Surface

Constrained motion was evaluated by bringing the end-effector into contact with the virtual wall and moving along the surface. Metrics were computed over the sustained-contact window from 20.50 s to 23.20 s, excluding entry and release transients. During this window, the mean penetration depth, computed from device–proxy separation, was 0.0492 m and the maximum penetration depth was 0.0512 m.

Force-command decomposition showed that the rendered force command was predominantly normal to the surface, with a median $|F_n|/|F_t|$ ratio of 17.72 and a 95th percentile ratio of 266.03. The sustained-contact logged force-command magnitude had a mean value of 24.68 N and a standard deviation of 0.72 N, corresponding to a coefficient of variation of 0.029.

The device and proxy trajectories are shown in Fig. 15. The proxy remains aligned with the virtual wall, while the device trajectory shows bounded displacement normal to the wall and motion along the tangential direction.

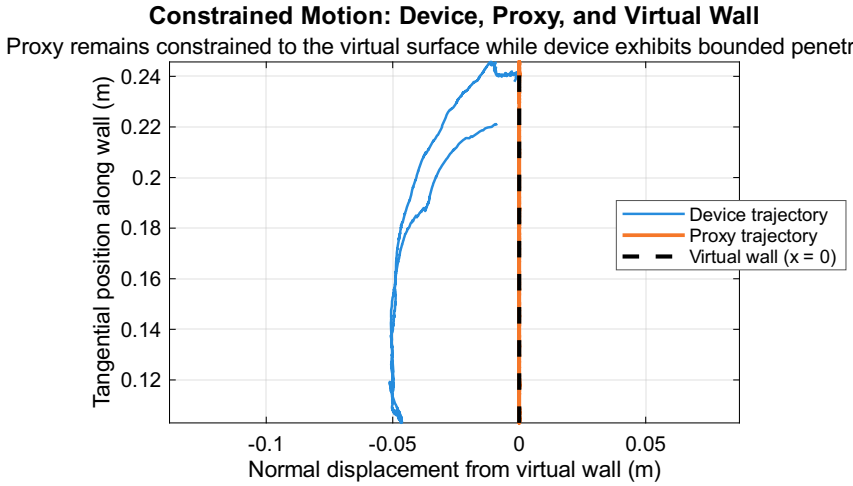


Fig. 15. Device and proxy trajectories during the analysed constrained-motion window from 20.50 s to 23.20 s. The proxy remains constrained to the wall while device normal displacement stays bounded.

Across the motor-enabled hardware tests, penetration remained bounded and logged force-command behaviour remained approximately consistent with the implemented virtual coupling model.

6 Discussion

This section interprets the main constraints on haptic performance: communication latency, delayed contact behaviour, hardware limitations, and design trade-offs. The emphasis is on what the results allow the system to claim, what they do not yet support, and how these limitations motivate the future work in Section 7.2.

6.1 Communication and Latency Constraints

The central finding from the communication evaluation is that transport delay, measured at 15.60–15.67 ms mean round-trip with 95th and 99th percentiles around 16.2–16.4 ms, is the dominant timing limitation. Host-side processing latency was much lower (mean 0.299 ms, 99th percentile 2.04 ms), so computation was not the bottleneck for the nominal 1 kHz local haptic/device loops. This delay bounds achievable stiffness because phase lag reduces stability margin and can inject artificial energy into the sampled-data loop [16]; the realised system therefore renders compliant rather than rigid contact.

The ST-Link virtual COM interface was convenient, but the burst-like delivery pattern and packet loss at low transmission rates suggest buffering and scheduling overhead somewhere in the USB

virtual-COM path, including the ST-Link interface, Windows serial stack, and host-side scheduling. The present measurements identify the path-level limitation but do not isolate which stage contributes most delay. The system should therefore be interpreted as a high-rate local renderer operating over delayed sampled state, rather than as a true 1 kHz end-to-end haptic loop. Snapshot channels prevent stale-state backlog, but cannot remove transport delay or guarantee 1 ms physical state freshness. This directly motivates the future communication work in Section 7.2, where the ST-Link path should be compared against lower-latency transport options before stronger responsiveness claims are made.

6.2 Local Contact State Limitation

The nominal 1 kHz haptic loop did not evolve a complete local contact state independently of slower simulation and communication layers. It consumed latest-available snapshots, then recomputed proxy/contact information each cycle. The renderer could execute at high frequency, but the physical meaning of each update was still bounded by snapshot freshness and communication delay.

A stronger architecture would maintain persistent proxy and local contact state inside the high-rate loop. Rather than re-projecting the proxy from scratch, the proxy would evolve continuously subject to contact constraints, preserving contact history, tangential motion, and release behaviour. This is closer to constraint-based proxy or god-object approaches [3], [12], [15] and to multilevel haptic rendering, where slow global updates are combined with a fast local model [3]. This limitation explains why future work should move either the full haptic renderer or a reduced local contact model closer to the device, rather than only increasing host-side update frequency.

6.3 Observed Contact Behaviour Under Delay

Despite imperfect communication timing, the tested interactions remained bounded because snapshot channels used the newest available state, thread separation avoided cross-subsystem blocking, damping countered delayed feedback, and embedded torque rate limiting prevented abrupt torque transitions. Watchdog activation also removed torque during communication interruptions (Section 5). These choices were effective for a prototype because they favoured conservative behaviour over maximum stiffness.

These mechanisms address the passivity concerns identified by Colgate and Schenkel [16] in a prac-

tical rather than formal way. The results show bounded behaviour in the tested cases, not a passivity proof. Contact remained controllable without divergent motion, and this is an important result given the measured 15–16 ms communication delay. However, the compliant feel was a direct consequence of delayed feedback; rigid contact cannot be achieved under the measured latency conditions by force-model tuning alone. The future-work route is therefore not simply to increase virtual stiffness, but to reduce effective delay, maintain local contact state, and add calibrated sensing so that stability and force fidelity can be assessed quantitatively.

6.4 Hardware Limitations

The hardware results must be interpreted within clear prototype constraints. The initial design targeted STM32 B-G431-ESC1 motor drivers, but integration with the selected magnetic encoders proved unreliable. SimpleFOC Mini modules provided a more robust integration path, but left torque control effectively open-loop with respect to current. Commanded torque therefore cannot be interpreted as calibrated physical torque; the hardware evaluation is primarily evidence of logged command consistency and qualitative interaction capability. This does not affect communication, timing, and bounded-contact results, which do not depend on absolute force calibration.

The GT2 belt reduction improved base-joint torque but slipped under high load, even after adding an adjustable idler pulley, probably because 3D-printed pulleys did not provide manufactured tooth-profile precision. Actuator torque limits would still bound force output even without slip, so both the transmission and actuator sizing constrain renderable stiffness. One actuator also failed during development, so final hardware validation used single-joint actuation with two-joint sensing. Conclusions about full 2-DOF force rendering therefore remain provisional. The corresponding future-work requirement is clear: restore two-axis actuation, add phase-current or external force measurement, and use a lower-backlash transmission before claiming calibrated multi-axis force feedback.

6.5 Design Trade-Offs

The system reflects a trade-off between bounded contact behaviour, responsiveness, and force fidelity. Damping and torque rate limiting avoided abrupt or divergent responses, but reduced perceived stiffness. The measured limiter of approximately 55 N m s^{-1} implies an approximate 0.13 s rise time to reach the 7 N m torque range, so hard contacts feel gradual rather than instantaneous.

Contact release could also feel slightly jerky, likely due to abrupt proxy-state changes, filtered velocity lag, or delayed torque commands. This shows why higher-fidelity rendering cannot be obtained by improving a single subsystem in isolation. Communication latency, contact-state evolution, actuator dynamics, transmission behaviour, and safety limits all interact, and the future work in Section 7.2 should therefore be treated as a combined architecture upgrade rather than as independent refinements.

6.6 Project Management and Development Process

The final Gantt chart in Appendix B reflects an adaptive process shaped by integration risk rather than a simple linear delivery plan. The project began with software-first development, allowing the host architecture, simulation structure, and communication approach to be explored before the final device was complete. As the physical prototype matured, hardware and software work then became increasingly parallel, with mechanical construction, embedded firmware, host communication, and simulation development progressing together. This was important because the project contained several high-risk interfaces: magnetic encoder alignment, motor-driver integration, USB serial communication, real-time host threading, and physical force-feedback safety.

The largest schedule deviation came from motor-driver development. The initial plan used STM32 B-G431-ESC1 drivers because they offered a more attractive route to current sensing and quantitative torque control. In practice, integrating those drivers with the selected encoder arrangement proved unreliable within the available time. The decision to pivot to SimpleFOC Mini modules reduced the achievable force-control fidelity, but it preserved the system-level aim by producing a working embedded platform for sensing, command reception, torque limiting, and watchdog safety. This was a trade-off between ideal subsystem performance and delivering an integrated prototype that could be tested end to end.

Risk management therefore focused on staged validation. Communication tests were separated from motor-enabled testing, allowing packet rate, burst behaviour, round-trip delay, and host-side processing latency to be characterised without exposing the user to actuator risk. Safety mechanisms were also validated before relying on the device for contact experiments: host-side force-command saturation, embedded torque limiting, torque slew-rate limiting, and watchdog torque removal all reduced the chance that software or communication faults would produce unsafe motion. This sequencing meant that even when final force-feedback testing became constrained, the communication and safety results remained usable.

A second major deviation occurred when one actuator failed during development. Rather than treating this as a complete project failure, the evaluation scope was narrowed to single-axis actuation with two-axis position sensing. This preserved evidence for the most important claims: the device could still render bounded contact in the tested cases, the host was not the dominant computation bottleneck, and the logging framework could distinguish command-side behaviour from physical limitations. At the same time, the limitation was carried through the report so that full 2-DOF force-feedback performance was not overstated.

Overall, the project management outcome was successful because the plan protected the central deliverable while allowing technical scope to adapt. The final prototype is less force-calibrated than originally intended, but the staged approach produced a defensible evaluation of the integrated haptic pipeline and a clear explanation of what must be improved next.

7 Conclusions and Future Work

This section summarises the extent to which the project objectives were achieved and identifies the main directions for future work.

7.1 Conclusions

This project designed, built, and evaluated a low-cost modular haptic rendering system combining a two-degree-of-freedom input device with custom simulation and visualisation software. At prototype level, user motion was sensed, transmitted to the host, coupled to a virtual environment, and returned to the device as actuator torque commands. The architecture also separated hardware communication, haptic rendering, physics simulation, and visualisation.

The most important technical result is that bounded haptic interaction was still achieved in the tested cases despite a measured round-trip communication delay of 15.60–15.67 ms. This latency prevents the system from being described as a true 1 kHz end-to-end haptic loop, even though selected local subsystems achieved nominal 1 kHz operation and host-side processing latency remained low. The results therefore separate two claims: the host was fast enough to compute and publish rendered-force commands locally, but the physical interaction was limited by delayed state freshness, actuator capability, transmission behaviour, and the absence of calibrated force or current sensing.

The project met the main evaluation objectives defined in Section 1. The sub-£200 interface was constructed with joint sensing and motor command capability, although final force-feedback validation was limited to one actuated joint after actuator failure. Communication tests demonstrated nominal 1 kHz packet generation and quantified packet rate, burst-like delivery, packet loss, out-of-order events, round-trip latency, and host-side processing delay. Motor-enabled tests showed bounded contact and consistent logged force-command behaviour in the tested interactions. Safety behaviour was supported by host-side force-command saturation, embedded torque limiting, torque slew-rate limiting, and watchdog torque removal.

The project therefore demonstrates an integrated low-cost haptic rendering pipeline, but it does not demonstrate calibrated physical force output or full two-axis force-feedback performance. Its strongest contribution is the evidence that useful bounded contact can be obtained on constrained hardware with a delayed communication path, provided that the system is designed around damping, saturation, latest-snapshot communication, and actuator-side safety limits. The main insight is that high-rate local computation and an accurate command-side force model are not sufficient on their own: communication delay, actuator capability, transmission behaviour, and the absence of calibrated physical sensing jointly determine achievable haptic fidelity. Stronger claims about haptic transparency, quantitative force fidelity, or training effectiveness would require lower-latency contact control and calibrated physical sensing.

7.2 Future Work

The first priority is to reduce effective actuator-side latency. Future work should compare the current ST-Link virtual COM pathway with lower-latency links such as direct UART through a dedicated USB–UART bridge, native USB transfer, CAN bus, or Ethernet-based communication. This comparison should use the same packet sequence numbering and timing logs developed in this project so that mean latency, jitter, burst delivery, dropped packets, and stale-state events can be compared directly. The aim would not simply be higher throughput, but lower and more predictable state age at the instant the actuator command is applied.

Future work should also move more contact behaviour closer to the high-rate loop. A stronger multi-rate architecture would maintain a reduced local contact model at 1 kHz, while slower physics, rendering, or application layers update the broader scene. The local model could store the currently contacted surface, proxy state, penetration history, and tangential motion, instead of re-computing

contact from delayed snapshots each cycle. Moving the proxy/contact evolution onto the embedded controller would further reduce dependence on delayed host–device exchanges, at the cost of simplifying the embedded contact model using local planes, SDF patches, virtual fixtures, or reduced contact geometry. This would make the system closer to established proxy or god-object rendering approaches while preserving the modular host simulation.

Hardware development should focus on calibrated sensing and restored two-axis actuation. Phase-current sensing and external force measurement would allow quantitative physical force validation and more accurate torque control, while a lower-backlash transmission such as a capstan drive would reduce belt slip. With calibrated sensing, the logged rendered-force command could be compared against actual output force rather than only command-side behaviour. This would support identification of actuator bandwidth, friction, backlash, and torque constant errors, and would allow the controller to be tuned against physical force fidelity rather than perceived contact quality alone.

Control development should then use the improved sensing and transmission model to move beyond the conservative damping-based prototype. Incorporating task-space dynamics would allow the manipulator inertia and kinematic mapping to be compensated more explicitly during interaction. More advanced impedance or admittance control could also be tested once physical force or current feedback is available. These changes would need to be introduced alongside passivity-aware limits, because increasing stiffness without reducing delay or adding damping would reintroduce the stability risks identified in the discussion.

Further software work should improve contact continuity and evaluation. Constraint-based proxy updates, smoother contact entry/release, velocity-state blending, and stale-command rejection would address force-direction inconsistencies during large penetration. Integration with robotic simulation platforms such as NVIDIA Isaac Sim, MATLAB/Simulink, or ROS-based environments would make it possible to test more realistic manipulation scenarios while keeping the low-cost device as the physical interface. Finally, force measurement, user studies, and task-based metrics should be used to assess whether the system improves training outcomes rather than only demonstrating technically bounded haptic interaction.

References

[1] G. S. Giri, Y. Maddahi, and K. Zareinia, "An Application-Based Review of Haptics Technology," *Robotics*, vol. 10, no. 1, p. 29, Feb. 5, 2021, ISSN: 2218-6581. DOI: 10.3390/robotics10010029 Accessed: Apr. 16, 2026. [Online]. Available: https://www.mdpi.com/2218-6581/10/1/29 (cited on pp. 1, 4, 7).

[2] M. Bergholz, M. Ferle, and B. M. Weber, "The benefits of haptic feedback in robot assisted surgery and their moderators: A meta-analysis," *Scientific Reports*, vol. 13, no. 1, p. 19 215, Nov. 6, 2023, ISSN: 2045-2322. DOI: 10.1038/s41598-023-46641-8 Accessed: Apr. 16, 2026. [Online]. Available: https://www.nature.com/articles/s41598-023-46641-8 (cited on p. 1).

[3] D. C. Ruspini, K. Kolarov, and O. Khatib, "The haptic display of complex graphical environments," in *Proceedings of the 24th Annual Conference on Computer Graphics and Interactive Techniques - SIGGRAPH '97*, Not Known: ACM Press, 1997, pp. 345–352, ISBN: 978-0-89791-896-1. DOI: 10.1145/258734.258878 Accessed: Apr. 16, 2026. [Online]. Available: http://portal.acm.org/citation.cfm?doid=258734.258878 (cited on pp. 1, 5, 13, 30).

[4] R. Adams and B. Hannaford, "Stable haptic interaction with virtual environments," *IEEE Transactions on Robotics and Automation*, vol. 15, no. 3, pp. 465–474, Jun. 1999, ISSN: 1042296X. DOI: 10.1109/70.768179 Accessed: Apr. 16, 2026. [Online]. Available: http://ieeexplore.ieee.org/document/768179/ (cited on pp. 1, 6, 12).

[5] M. O. Martinez et al., "3-D printed haptic devices for educational applications," in *2016 IEEE Haptics Symposium (HAPTICS)*, Apr. 2016, pp. 126–133. DOI: 10.1109/HAPTICS.2016.7463166 Accessed: Apr. 16, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/7463166 (cited on pp. 2, 7).

[6] M. O. Martinez et al., "Open source, modular, customizable, 3-D printed kinesthetic haptic devices," in *2017 IEEE World Haptics Conference (WHC)*, Jun. 2017, pp. 142–147. DOI: 10.1109/WHC.2017.7989891 Accessed: Apr. 16, 2026. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/7989891 (cited on pp. 2, 7).

[7] A. Lavatelli, M. Bordegoni, and F. Ferrise, "Design of an open-source low cost 2DOF haptic device," Sep. 2014. DOI: 10.1109/MESA.2014.6935574 (cited on pp. 2, 7).

[8] "Geomagic touch haptic device," Accessed: Apr. 16, 2026. [Online]. Available: https://shop.gomeasure3d.com/products/geomagic-touch-haptic-device (cited on p. 4).

[9] “Geomagic touch X haptic device,” Accessed: Apr. 16, 2026. [Online]. Available: https://shop.gomeasure3d.com/products/geomagic-touch-x-haptic-device (cited on p. 4).

[10] “Sales inquiries,” Accessed: Apr. 16, 2026. [Online]. Available: https://www.forcedimension.com/contact/sales (cited on p. 4).

[11] “Inverse3 X,” Accessed: Apr. 16, 2026. [Online]. Available: https://www.haply.co/inverse3x (cited on p. 4).

[12] C. Zilles and J. Salisbury, “A constraint-based god-object method for haptic display,” in *Proceedings 1995 IEEE/RSJ International Conference on Intelligent Robots and Systems. Human Robot Interaction and Cooperative Robots*, vol. 3, Pittsburgh, PA, USA: IEEE Comput. Soc. Press, 1995, pp. 146–151, ISBN: 978-0-8186-7108-1. DOI: 10.1109/IROS.1995.525876 Accessed: Apr. 16, 2026. [Online]. Available: http://ieeexplore.ieee.org/document/525876/ (cited on pp. 5, 13, 30).

[13] O. Khatib, “A unified approach for motion and force control of robot manipulators: The operational space formulation,” *IEEE Journal on Robotics and Automation*, vol. 3, no. 1, pp. 43–53, Feb. 1987, ISSN: 2374-8710. DOI: 10.1109/JRA.1987.1087068 Accessed: Apr. 16, 2026. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/1087068 (cited on p. 5).

[14] C. Ericson, *Real-Time Collision Detection*, 0th ed. CRC Press, Dec. 22, 2004, ISBN: 978-0-08-047414-4. DOI: 10.1201/b14581 Accessed: Apr. 16, 2026. [Online]. Available: https://www.taylorfrancis.com/books/9780080474144 (cited on pp. 5, 12).

[15] K. Salisbury and C. Tarr, “Haptic Rendering of Surfaces Defined by Implicit Functions,” in *Dynamic Systems and Control*, Dallas, Texas, USA: American Society of Mechanical Engineers, Nov. 16, 1997, pp. 61–67, ISBN: 978-0-7918-1824-4. DOI: 10.1115/IMECE1997-0378 Accessed: Apr. 16, 2026. [Online]. Available: https://asmedigitalcollection.asme.org/IMECE/proceedings/IMECE97/18244/61/1138197 (cited on pp. 5, 13, 30).

[16] J. E. Colgate and G. G. Schenkel, “Passivity of a class of sampled-data systems: Application to haptic interfaces,” *Journal of Robotic Systems*, vol. 14, no. 1, pp. 37–47, 1997, ISSN: 1097-4563. DOI: 10.1002/(SICI)1097-4563(199701)14:1<37::AID-ROB4>3.0.CO;2-V Accessed: Apr. 18, 2026. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/%28SICI%291097-4563%28199701%2914%3A1%3C37%3A%3AAID-ROB4%3E3.0.CO%3B2-V (cited on pp. 6, 29, 30).

[17] L. Ramadoss and J. Y. Hung, “A study on universal serial bus latency in a real-time control system,” in *2008 34th Annual Conference of IEEE Industrial Electronics*, Nov. 2008, pp. 67–72. DOI: 10.1109/IECON.2008.4757930 Accessed: May 4, 2026. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/4757930 (cited on p. 6).

[18] T-Motor, *GB2208 gimbal type brushless motor*, 2026. [Online]. Available: https://store.tmotor.com/product/gb2208-gimbal-type.html (cited on p. 9).

[19] ams AG, *AS5048A/AS5048B magnetic rotary encoder (14-bit angular position sensor)*, v1-11, manual, ams AG, Jan. 2018. Accessed: Apr. 16, 2026. [Online]. Available: https://ams-osram.com/products/sensors/position-sensors/ams-as5048a-high-resolution-position-sensor (cited on p. 9).

[20] ams AG, *AS5600 12-bit programmable contactless potentiometer*, v1-06, manual, ams AG, Jun. 2018. Accessed: Apr. 16, 2026. [Online]. Available: https://ams-osram.com/products/sensor-solutions/position-sensors/ams-as5600-position-sensor (cited on p. 9).

[21] “NUCLEO-G431RB | Product - STMicroelectronics,” Accessed: Apr. 16, 2026. [Online]. Available: https://www.st.com/en/evaluation-tools/nucleo-g431rb.html (cited on p. 10).

[22] A. Skuric et al., “SimpleFOC: A field oriented control (FOC) library for controlling brushless direct current (BLDC) and stepper motors,” *Journal of Open Source Software*, vol. 7, no. 74, p. 4232, 2022. DOI: 10.21105/joss.04232 [Online]. Available: https://doi.org/10.21105/joss.04232 (cited on p. 10).

[23] P. Vas, *Sensorless Vector and Direct Torque Control*. Oxford University Press, May 14, 1998, ISBN: 978-0-19-856465-2. DOI: 10.1093/oso/9780198564652.001.0001 Accessed: Apr. 16, 2026. [Online]. Available: https://doi.org/10.1093/oso/9780198564652.001.0001 (cited on p. 10).

Appendices

A Preliminary Project Proposal

This appendix records the initial project outline submitted at the start of the work, providing context for how the project scope evolved during development.

Design and Development of a Low-Cost 2-DOF Haptic Training Interface for Robotic Manipulator Simulation

Tyler Ward

October 2025

1 Background and Motivation

Robotic manipulators are increasingly used in manufacturing, surgical, and remote handling applications. However, operator training often relies on visual feedback alone, limiting spatial understanding and increasing the risk of errors when transitioning to physical systems.

Haptic feedback provides tactile and force based sensations that improve the realism and safety of training by allowing users to feel resistance and contact forces. Commercial haptic devices are often expensive and closed source, which makes them unsuitable for flexible training or academic development.

This project aims to create a low cost modular 2 DOF haptic input device that can send user motion commands to a simulated robotic manipulator and receive realistic force feedback. The system will be designed to support future integration with real robotic hardware, providing an accessible and scalable training platform.

2 Aim

To design, prototype and evaluate a compact low cost two degree of freedom haptic interface that can communicate in real time with a robotic manipulator simulation and provide accurate position sensing and responsive force feedback to the user.

3 Objectives

1. **Literature and Technology Review**
   A review will be carried out to identify existing haptic devices and relevant control methods. This will guide the selection of motors, encoders, and actuation strategies suitable for a low cost modular interface.
2. **Physics Simulation Development**
   A physics based simulation environment will be designed to model manipulator dynamics, haptic interactions, and constraint behavior.
3. **Hardware Design**

1

Fig. 16. Original project outline as submitted at the start of the project

B Gantt Chart

This appendix shows the final project schedule and the main development phases completed during the project.

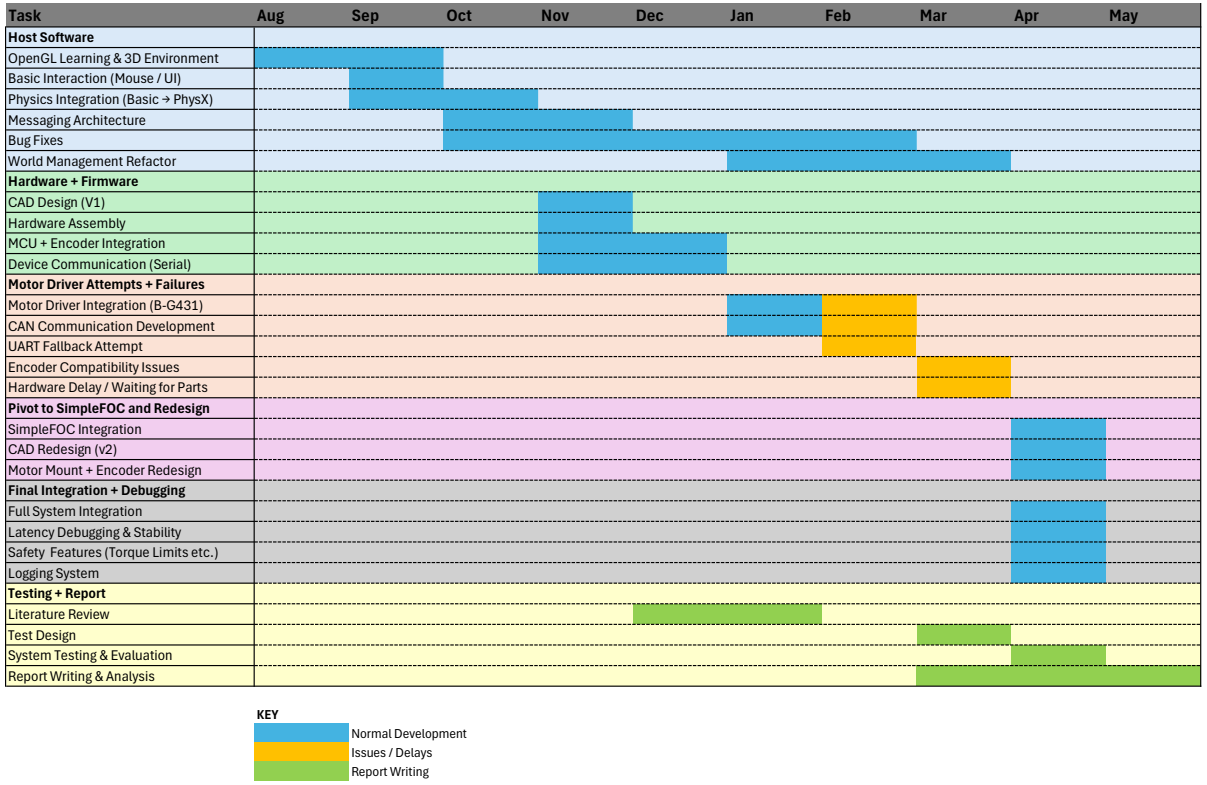


Fig. 17. Final Gantt chart showing the execution of the project over time. The chart reflects the iterative development process, including overlapping software and hardware work, as well as deviations from the original plan due to motor driver integration issues, hardware delays, and subsystem integration challenges.

C Risk Register and Health & Safety Risk Assessment

This appendix contains the project risk assessment used to identify safety, technical, and delivery risks.

MANCHESTER 1824
The University of Manchester

RISK REGISTER FOR 3rd YEAR PROJECT

Project Title:	**A Real-Time Haptic Rendering System Integrating Physics-Based Simulation and OpenGL Visualisation**		Submission Date:	1/05/2026	
Student Name:	11349510				

Project Risk	Severity			Potential			Score (Severity x Potential)	Mitigation Measures
	L	M	H	L	M	H	L=1, M=2, H=3	
Electrical faults during wiring/testing (short circuit, incorrect connections)	X				X		2	Use low-voltage systems; power off before modifying wiring; check connections before powering; use insulated connectors
Injury from moving mechanical parts (rotating joints, links)		X			X		4	Limit actuator torque; avoid contact during operation; test at low power; implement watchdog shutdown
Actuator overheating during prolonged operation		X		X			2	Monitor device temperature; limit runtime; allow cooling periods; avoid sustained high torque
Mechanical failure (belt slip, loose components)		X			X		4	Secure all components; inspect before testing; avoid excessive loading; operate within tested limits
Control instability causing unexpected motion or oscillations			X		X		6	Use conservative control gains; include damping; implement torque limits and rate limiting; test incrementally
Communication failure (loss of command stream)		X			X		4	Implement watchdog timeout to zero torque; validate communication before operation
Software bugs causing incorrect behaviour		X			X		4	Incremental testing; logging and debugging; isolate subsystems during development
Component failure (e.g. actuator failure during project)		X			X		4	Design for partial operation; maintain flexibility in testing; prioritise core functionality
Use of tools during assembly (cuts, slips)	X				X		2	Use appropriate tools; follow workshop safety procedures; maintain tidy workspace
Extended computer use (eye strain, fatigue)	X					X	3	Take regular breaks; maintain ergonomic posture; adjust screen setup

Fig. 18. Project Risk Assessment

D CPD Log

This appendix records the continuing professional development activities undertaken during the project.

Continuing Professional Development Log

Name: *Tyler Ward*

Current and recent CPD activity:

CPD Activity Title	Description	Dates	CPD Hours
OpenGL and Real-Time Rendering	Self-taught OpenGL, shaders, and 3D rendering concepts using online resources.	Aug 2025 – Oct 2025	~20 hours
Haptic Rendering Methods	Read academic papers on virtual coupling and proxy-based haptics to understand force rendering approaches.	Aug 2025 – Jan 2026	~20hrs
BLDC Motor Control (SimpleFOC)	Learned to use SimpleFOC library for BLDC motor control and torque-mode operation.	Jan 2026 – Mar 2026	~10 hours
PhysX Simulation Integration	Learned how to integrate and use a physics engine (PhysX) for real-time simulation.	Jan 2026 – Feb 2026	~10hrs
Hardware Integration and Iterative Design	Gained practical experience designing and refining a mechanical system with motors and encoders.	Nov 2025 – Mar 2026	~20hrs
Working with a Large Codebase	Gained experience developing and managing a multi-component C++ codebase, including structuring modules, debugging across subsystems, and maintaining clarity in a multi-threaded architecture.	Aug 2025 – Apr 2026	~50 hrs
Technical Report Writing and Literature Review	Developed skills in structuring a technical report and conducting a focused literature review, including synthesising multiple sources, strengthening technical claims with references, and improving clarity and precision in written communication.	Dec 2025 – May 2026	~25 hrs

Planned and Future CPD activity:

CPD Activity Title	Description	Skills addressed	Dates
Technical Research and Literature Skills	Further develop ability to read and interpret technical research papers, including understanding methodologies, extracting key insights, and critically evaluating approaches.	Research skills, critical analysis, technical understanding	Summer 2026
Long-Term Project Planning	Improve ability to plan and manage longer-term technical projects, including setting realistic milestones, managing uncertainty, and adapting plans as challenges arise.	Project management, planning, problem-solving	Summer 2026

Fig. 19. CPD Log documenting skills and knowledge acquired during the project.

E Mechanical Design Drawing

The mechanical design drawing documents the main link dimensions, actuator placement, and encoder arrangement used by the physical prototype.

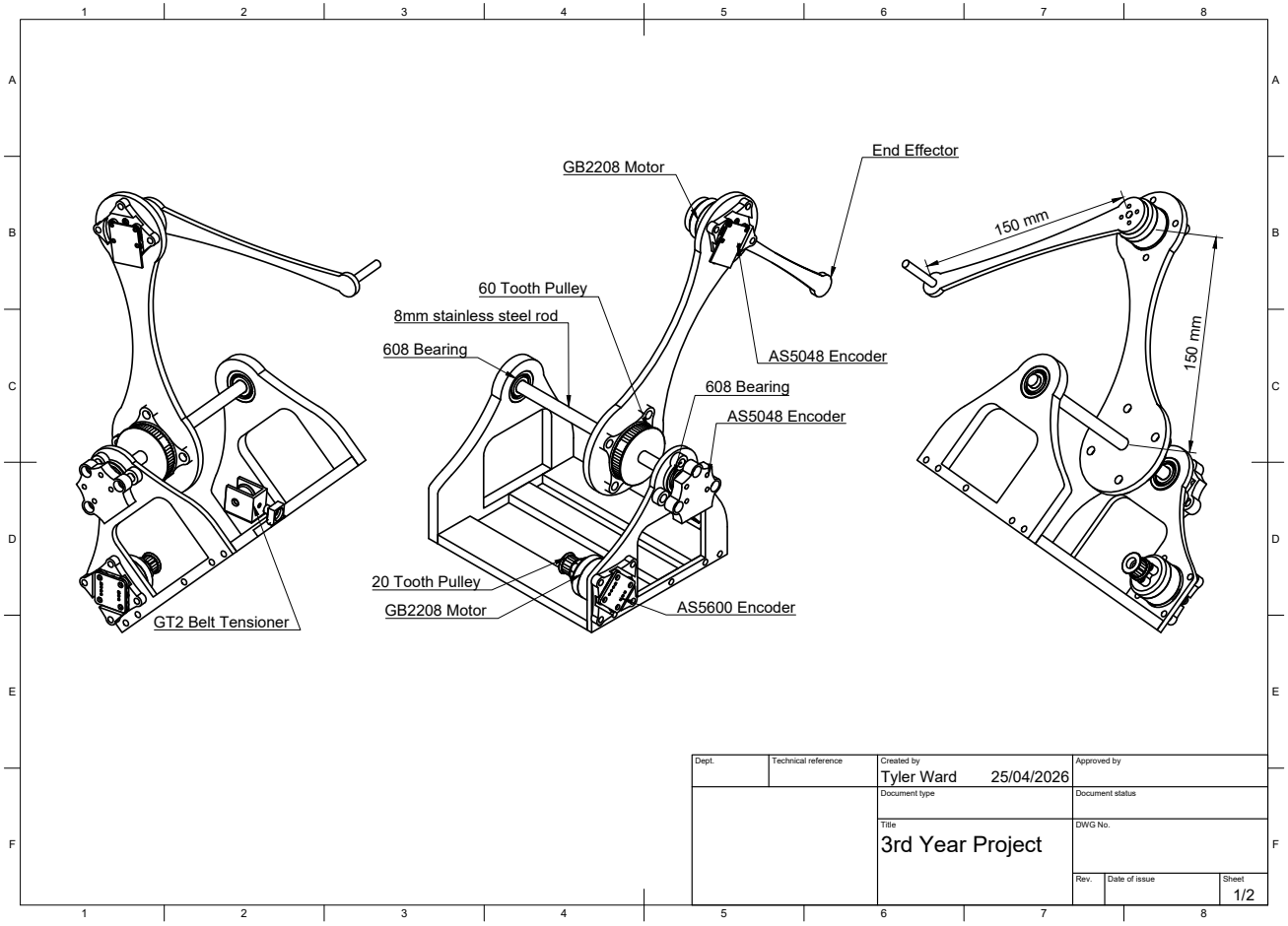


Fig. 20. CAD assembly of the 2-DOF haptic interface illustrating link geometry, actuator configuration, belt-driven base joint, and encoder placement. Link dimensions are annotated to define the kinematic model used for forward kinematics and Jacobian-based torque mapping.

F Source Code and Repository Information

The project source code is available from the GitHub repository:

`https://github.com/TylerWardUOM/3rd-yr-project`

This repository contains the software, firmware, analysis scripts, and documentation used to develop and evaluate the low-cost haptic rendering prototype. The main application is a C++17 Windows program built with CMake. It integrates OpenGL/ImGui visualisation, PhysX-based simulation, proxy-based haptic rendering, serial communication, runtime logging, and torque-command generation for the physical device. The firmware directory contains the microcontroller code for encoder reading, packet generation, torque-command reception, actuator command output, slew-rate limiting, and watchdog safety behaviour.

The repository is organised around the main development artefacts rather than only the final report. The `src/` and `include/` directories contain the host-side application, including the renderer, physics engine wrapper, haptic engine, message-bus channels, serial link, and device adapter. The `firmware/` directory contains the embedded controller implementation. The `data/` and analysis-related files contain experimental logs and scripts used to generate the communication, timing, safety, and haptic interaction results reported in the dissertation. The report source, figures, references, appendices, and supporting project documentation are stored in the report documentation folder.

Two executable entry points are most relevant. The main haptic application runs the integrated visualisation, physics, haptic rendering, and device-control pipeline. The `serial_tests` utility performs standalone packet-rate, integrity, and round-trip latency tests using the same packet framing and parsing approach as the main application. These tools were used to separate host-side processing time from transport delay and to characterise the burst-like delivery observed through the USB virtual-COM communication path.

The repository should be read as supporting evidence for the design and evaluation described in the report. It documents the implemented architecture, the runtime logging framework, the experimental test utilities, and the limitations of the prototype. In particular, the code records that final motor-enabled validation used single-axis force feedback after actuator failure, and that logged force-command and torque-command data were used because no calibrated physical force sensor or phase-current measurement was available.